

<TITULO TESIS>

<AUTORES>**MBIT School**c/ Serrano 213
28016, MadridTel: 91 504 86 00
info@mbitschool.com

MBIT School	0
Resumen ejecutivo.....	2
Motivación del proyecto	3
Caso de uso	3
Objetivos	3
Hipótesis iniciales	3
Problemas a resolver	3
Riesgos y problemas.....	4
Referencias.....	4
Bibliografía	4
Estado del arte.....	6
Análisis de soluciones existentes	6
Limitaciones conocidas	6
Justificación del desarrollo	6
Solución técnica.....	7
Requerimientos	7
Arquitectura y elementos	7
Infraestructura	7
Volumetría y escalabilidad	7
Costes operativos	8
Fuentes de datos	8
Productos y librerías necesarios	9
Plan de trabajo.....	10
Planificación	10
Tareas e hitos	10
Perfiles y esfuerzos.....	10
Resultados.....	11
Resultados tangibles.....	11
Validación de resultados	11
Conclusiones.....	11
Sigüientes pasos.....	11
Código fuente	12
GitHub y estructura de repositorio	12
Instalación.....	12
Guía rápida de ejecución	12
Librerías y versiones	13

Resumen ejecutivo

Resumen de una página del proyecto.

El estudiante debe ofrecer una visión sintética del proyecto: cuál es el problema abordado, qué solución se propone, cuál ha sido el enfoque técnico utilizado y cuáles son los resultados o conclusiones más relevantes. Debe permitir al lector comprender el propósito y el alcance del proyecto sin necesidad de leer el documento completo.

Casos de uso y objetivos

Introducción

Desarrollar una breve introducción al proyecto.

Debe explicarse por qué el estudiante eligió este tema, qué necesidades reales detectó y qué valor aportaría la solución propuesta. Se espera una reflexión personal y profesional apoyada en contexto del sector, tendencias o problemas existentes.

Caso de uso

Aquí el estudiante debe describir el escenario concreto donde la solución será aplicada: quiénes serían los usuarios, qué problema experimentan, cómo interactuarían con la solución y qué beneficios se obtendrían. Debe ser un caso realista, bien definido y alineado con el objetivo general del proyecto.

Objetivos

Se deben enumerar los objetivos generales y específicos del proyecto, indicando qué se pretende conseguir desde los puntos de vista técnico, funcional y de negocio. Los objetivos deben ser medibles y alcanzables dentro del alcance del proyecto.

Hipótesis iniciales

El estudiante debe enunciar las suposiciones de partida que guían el proyecto: qué se espera que ocurra, qué comportamientos deberían observarse, y sobre qué premisas se construye la solución. Las hipótesis permitirán evaluar posteriormente si los resultados han validado o refutado dichas expectativas.

Problemas a resolver

En esta sección el estudiante debe detallar los desafíos o limitaciones actuales del dominio del proyecto: dificultades del usuario, barreras técnicas, ineficiencias o carencias identificadas. Este apartado debe conectar directamente con la motivación y los objetivos.

Riesgos y problemas

Aquí se deben identificar los riesgos potenciales del proyecto, tanto técnicos como organizativos: dependencia de datos, limitaciones de hardware, incertidumbre en el rendimiento, complejidad de modelos, privacidad, costes o plazos. Para cada riesgo se sugiere incluir una estrategia de mitigación.

Referencias

El estudiante debe incluir las fuentes de información utilizadas para fundamentar la propuesta: documentación técnica, artículos, soluciones existentes, benchmarks, papers académicos o publicaciones relevantes para el dominio.

Bibliografía

Aquí se deben listar todas las fuentes citadas a lo largo del documento, siguiendo un formato consistente (APA, IEEE, etc.). Incluye papers, documentación oficial, blogs técnicos, reportes o recursos utilizados durante el proyecto.

Estado del arte

Análisis de soluciones existentes

El estudiante debe evaluar herramientas, productos o investigaciones similares, indicando sus fortalezas, debilidades y limitaciones. Esta comparación permite justificar la necesidad del proyecto y tomar decisiones de diseño informadas.

Limitaciones conocidas

Debe explicarse qué aspectos de las soluciones actuales no cubren adecuadamente el problema o presentan carencias. Esto refuerza la elección del enfoque adoptado y define el espacio donde el proyecto aporta valor.

Justificación del desarrollo

Se debe argumentar por qué el proyecto es necesario y qué ventajas aporta frente a las alternativas existentes. Aquí se explica cómo la solución propuesta mejora, complementa o supera las opciones actuales.

Solución técnica

Requerimientos

El estudiante debe describir los requisitos funcionales (qué debe hacer el sistema) y no funcionales (rendimiento, seguridad, escalabilidad, usabilidad). Estos requisitos guiarán las decisiones del diseño técnico.

Arquitectura y elementos

Se debe detallar la arquitectura del sistema, preferentemente con diagramas: componentes, módulos, flujos de datos, interacciones entre elementos, decisiones de diseño y justificaciones técnicas. Si utilizamos servicios cloud, procurar utilizar los iconos propios de cada cloud y seguir sus estándares de diseño.

Infraestructura

Aquí se describe el entorno necesario para ejecutar la solución: hardware, cloud, contenedores, GPU, servicios externos, almacenamiento, pipelines, etc. Debe incluir tanto la infraestructura actual como la prevista para un entorno escalado.

Volumetría y escalabilidad

El estudiante debe estimar cuánto volumen de datos, usuarios o peticiones se espera manejar y justificar cómo la solución puede escalar ante cargas mayores. Debe incluir análisis de performance, cuellos de botella y posibles mejoras.

Costes operativos

Se deben identificar los costes asociados al proyecto: uso de APIs, infraestructura cloud, almacenamiento, consumo de GPU, mantenimiento, etc. Debe incluir una estimación económica y posibles optimizaciones para reducir gastos. Apoyarse en una tabla para facilitar el desglose.

Fuentes de datos

En este apartado se especifican los tipos de datos utilizados: su origen, formato, calidad, volumen y tratamientos realizados (limpieza, normalización, etiquetado). Es crítico justificar la adecuación de los datos al caso de uso.

Productos y librerías necesarios

Aquí el estudiante debe listar todas las herramientas, librerías, frameworks y modelos utilizados, explicando por qué fueron elegidos y cómo contribuyen al sistema.

Plan de trabajo

Metodología

Debe incluir la metodología de desarrollo seguida (CRISP-DM, Agile, Kanban...), la organización del trabajo y la estructura general del proceso de construcción del proyecto.

Planificación

El estudiante debe mostrar el cronograma o timeline del proyecto, con las fechas de inicio y fin de cada fase, hitos importantes y dependencias. Una tabla o diagrama tipo Gantt es recomendable.

Tareas e hitos

Se deben describir las principales tareas del proyecto, el objetivo de cada una y qué entregables se generaron. Los hitos son puntos clave de avance o validación.

Perfiles y esfuerzos

Si el proyecto es grupal, deben especificarse los roles, responsabilidades y distribución del esfuerzo. En un proyecto individual, debe describirse el volumen de trabajo y la dedicación aproximada por áreas.

Resultados

Resultados tangibles

El estudiante debe mostrar entregables concretos: prototipos, modelos entrenados, dashboards, informes, documentos finales, interfaces, APIs o datasets generados.

Validación de resultados

Aquí deben evaluarse los resultados obtenidos frente a los objetivos e hipótesis iniciales. Se esperan métricas, comparativas, testing, ejemplos de uso real, limitaciones y análisis crítico.

Conclusiones

El estudiante debe sintetizar las aportaciones del proyecto, los aprendizajes, el impacto logrado y el grado de cumplimiento de los objetivos definidos al inicio.

Siguientes pasos

Debe describirse qué mejoras futuras podrían implementarse, qué ampliaciones son posibles, qué líneas de evolución existen y cómo podría escalar o industrializarse la solución.

Código fuente

GitHub y estructura de repositorio

El estudiante debe proporcionar el enlace al repositorio, explicar la estructura de carpetas, módulos y scripts, y describir brevemente qué contiene cada sección.

Instalación

Aquí se explica cómo instalar y configurar el proyecto: dependencias, librerías, entorno virtual, archivos de configuración, credenciales y pasos básicos.

Guía rápida de ejecución

Debe explicar al usuario cómo ejecutar la aplicación o prototipo en su forma más simple: comandos principales, parámetros necesarios y ejemplos.

Librerías y versiones

El estudiante debe detallar las versiones concretas de Python, librerías, modelos o frameworks utilizados para garantizar reproducibilidad y evitar incompatibilidades.